DEVOPS & CLOUD [SWE] · SCALER ACADEMY

$ cat notes.md

DevOps & Cloud [SWE] — Class 1 Notes

LECTURE TITLE	**Introduction to DevOps & Microservices**
DATE/TIME	**11 August 2026, 09:35 AM**
DURATION	**120 minutes**
INSTRUCTOR	**Nensi Ravaliya**
BIG IDEA	**One big app vs. many small apps — and why that single choice decides everything else you'll learn this course.**

WHAT'S INSIDE — 18 SECTIONS

00 ⚡ The 60-Second Version
01 🗓️ Housekeeping & Warm-Up
02 🎓 The Running Example: A College Placement Portal
03 🧱 Monolithic Architecture — "One Giant Box"
04 🧩 Microservices Architecture — "Many Small Shops"
05 🥊 Monolith vs Microservices — The Scoreboard
06 🌉 Why DevOps Even Exists
07 🧰 The DevOps Toolbox — All 13 Tools
08 🎒 The CS Fundamentals They'll Grill You On
09 🧭 The Instructor's Own Career Story
10 ♾️ The DevOps Lifecycle — 8 Stages in a Loop
11 💼 Job Roles — The Whole Menu
12 🙋 Q&A — Role Confusion, Cleared Up
13 🎮 Quiz Time — Team A vs. Team B vs. Team C
14 🥊 The Great Monolith Debate
15 🔁 Class Recap Checklist
16 📒 Cheat-Sheet — Everything Introduced Today
17 🗒️ Notes on the Transcript Itself

Rebuilt from the raw class transcript — nothing cut, just organised, de-noised and made readable. `theme: Sunrise`

00 ⚡ The 60-Second Version

Read this, and you already have 80% of the class in your head:

1. **A monolith is one box.** Login, jobs, payments, admin — all in one codebase, shipped as one thing. Change one button colour → rebuild and redeploy *everything*.
2. **Microservices are many small boxes.** Each feature is its own codebase, its own deploy. Payments can crash while login stays perfectly fine.
3. **Monolith is easier to *start*. Microservices are easier to *live with*.** That's the whole trade.
4. **DevOps is what makes "many small boxes" survivable.** 6 services = 6 builds, 6 deploys, 6 sets of logs. Nobody does that by hand, so you automate it: Docker → CI/CD → Cloud → Kubernetes → Terraform → Ansible → Monitoring.
5. **The lifecycle is a circle, not a line:** Plan → Code → Build → Test → Release → Deploy → Operate → Monitor → back to Plan.
6. **DevSecOps in one word: "shift left."** Stop checking security at the very end; check it at every single step.

◆ ◆ ◆

01 🗒️ Housekeeping & Warm-Up

The traditional first-class tech gremlins

The first ~5 minutes were the instructor fighting her own setup — iPad not mirroring to the screen, "is it on?", "can you hear me?", trying screen-extend vs. screen-mirror, multiple devices connected, recording not starting. Classic day one. Eventually the iPad connected and became the digital whiteboard for the whole session.

🎤 SAID IN CLASS

Actual quote: *"Hi everyone, audible right? Everyone?"* — the official start of the course.

Who's teaching you

Day job	DevSecOps Engineer at a Fortune 500 company
Side gigs	Udemy instructor (~5 courses), YouTube channel, blogs on Medium + own platform, content creator / influencer / trainer
Topics she posts on	Cloud, DevOps, Machine Learning — and a lot of AI lately, "because it's the AI era"

Certifications	**11 total.** AWS: Solutions Architect + a brand-new advanced cert that only **30–40 people worldwide** hold (barely any study material exists for it yet). Azure: cybersecurity, DevOps Expert, Solutions Architect, Database Administrator, Developer, and AI certs.
Clouds explored	AWS, Azure, GCP, Oracle, Salesforce, IBM — "almost all of them"
Cloud experience	4 years, journey started with AWS
Current focus	DevOps, DevSecOps, MLOps

💡 GOOD TO KNOW

Her one-line career advice: *"If you want to survive in this industry, cloud knowledge alone is enough."* Everything else in DevOps stacks on top of that.

Batch background poll

She refuses to lecture blind, so she polled the room first:

Question	What the room said
Year / branch?	All 3rd–4th year, **all Computer Science**
Know Linux?	Most said yes — so she immediately tested it: *"Anyone know the* `chmod` *command?"* (her point: it's a command you touch **every single day** once you're on real servers)
How many projects?	Answers ranged from **1 project to 50+**. Genuinely wide spread.
Explored DevOps before?	Mixed — several had **never** touched it and joined this batch cold. She called that "nice courage."
Used AWS?	A few — **EC2** and **S3**
Have a portfolio?	A handful of hands

Course roadmap she promised

```
Phase 1 (revision)  →  Git · GitHub · Linux · Shell Scripting
Phase 2 (real DevOps) → Docker · architecture concepts
Phase 3 (cloud)     →  mainly AWS (biggest provider), some Azure
Phase 4 (bonus)     →  DevSecOps = DevOps + Security
```

◆ ◆ ◆

02 🎓 The Running Example: A College Placement Portal

Before any theory, she grounded the entire class in one relatable problem.

The pain: *"How many of you use LinkedIn? Applied for a job through it?"* Very few hands. And the ones who did? They either got an **"Unfortunately…"** email or **complete silence**.

The idea: so build your own **college placement portal** — students apply to companies, upload resumes, track their applications, move through interview rounds. Every architecture concept in this class gets explained through this one app.

🧠 PICTURE THIS

Why this example, specifically? Because a placement portal has one sneaky property: **the user base only grows.** Every year a new batch joins, and last year's batch becomes alumni whose data you *keep forever*. No one ever deletes. This single fact ends up deciding the architecture — you'll see it in §14.

She also quickly checked: *"SDLC — full form?"* → **Software Development Life Cycle.** Reason for the question: the DevOps lifecycle we cover later is basically SDLC with automation bolted on.

◆ ◆ ◆

03 📦 Monolithic Architecture — "One Giant Box"

The student definition (and it's a good start)

🎤 SAID IN CLASS

Student answer: *"Monolithic architecture — everything that we need in one single repository."*

Instructor: good, good. That's the right instinct.

The Scaler-website walkthrough

To make it concrete, she opened **scaler.com** live and pointed at the tabs:

```
Logo | Programs | Masterclass | EI Labs | Alumni | Resources | Placement Report | My Profile
```

Six or seven visible sections. In a monolith, **all** of that code lives in **one** repository — frontend, backend, database access, everything bundled together.

Now the painful part

Suppose you want to change *one thing* — the colour of the "Placement Report" button from blue to something else. Here's your day:

```
1. Developer changes one line of CSS
2. Pushes the code
3. Whole CI/CD pipeline runs — for the ENTIRE application
4. Whole application gets built
5. Whole application gets deployed
6. ...now, finally, you can see the new button colour
```

⚠️ WATCH OUT

The gotcha: you changed one character. You redeployed everything. And everything means every other feature also went through a build, a test cycle, and a deployment window — for a button colour.

Definition to remember: a monolith is when all six-to-seven components live inside **one repository / one codebase**, and any change — however tiny — means rebuilding and redeploying the whole thing.

◆ ◆ ◆

04 🧩 Microservices Architecture — "Many Small Shops"

The college-department analogy

This was the best explanation of the class, so here it is in full.

🧠 PICTURE THIS

Analogy: Think about your own college. There's an **admission** department, a **finance** department, **classrooms**, **HODs**, a **library** — many separate departments.

Now: do you walk into the **library** and say *"I need to pay my fee, where do I go?"* No. After paying at **finance**, do you say *"also, I don't like this faculty, please remove them"*? Also no.

You ask the **library** for books. You ask **finance** about fees. Each department has its own job, its own people, its own records. **That's microservices.**

The Zepto analogy

Second example, even closer to code:

- Every couple of days you open **Zepto** and add milk, curd, vegetables, fruits to your cart. **Adding to cart = one process**, built by one small team, living in one small codebase — **one microservice**.
- Then you **pay**. Payment is a *completely different* codebase, with *different* developers.

- Why separate? Because **payment has nothing to do with your login data**, and login has nothing to do with payment. Zero overlap. So why glue them together?

So an app is really many mini-apps

Inside one "application" you actually have several small applications:

Mini-app	Owns
User service	login, profile, user data
Payment service	transactions, billing
Management service	internal operations
Admin service	role-based access, permissions, onboarding

> 💡 GOOD TO KNOW
>
> **The admin point she made:** as a *user* you see one platform. As an *admin* you see a **totally different** one — who to permit, what roles to grant, and on an ed-tech platform, an entire flow for **onboarding instructors** that normal users never even know exists. Different audience → different service.

Different services, different tech stacks

And because they're separate, they don't even have to use the same language:

```
Frontend  →  React / Node.js       (the login page you see)
Backend   →  Java · Spring Boot · Quarkus   (microservices doing the work)
```

> ⚠️ WATCH OUT
>
> **Her point:** these two are things *"we cannot merge into one thing"* — and in a monolith you'd be forced to try.

The clean contrast

	Monolith	Microservices
Codebase	frontend + backend + database, all in **one** repo	**many** separate codebases
Running it	start the one thing	start frontend, start backend, connect DB, *then* the app is up

◆ ◆ ◆

05 🧱 Monolith vs Microservices — The Scoreboard

She flipped the question back at the class: *"Why can't we just use monolith? There's only one repo, changes happen in one place — why create five repositories for one website?"*

Answers the room shouted out:

Problem with monolith	Who nailed it
Scaling	first hand up — "nice, good"
Deployment issues	"perfect"
Distribution of team	✅
Management	✅

Then the reverse: *"Even if there are five repos and five developers, why bother with microservices?"*

Reason for microservices	Note
Individually deployable	ship payments without touching login
One service down ≠ site down	*"Exactly. This was the one issue we faced in monolith."*

The failure story that makes it click

In a monolith: if the frontend doesn't come up, your website is **dead** — even though the backend is working perfectly. And if 10,000 users arrive at an app built for 2,000? *"Surely it will fail."*

In microservices: the frontend loads fine, and the broken part shows up as a specific error instead of a dead site:

What you see	What it actually means
502 / 503	Gateway error → **backend / engine problem**
404	route/resource not found
Raw JSON on screen (e.g. `payment found`)	**frontend** problem — it got data but couldn't render it

✅ LOCK THIS IN

Remember this: those 502/503 errors you've cursed at on real websites are *proof* the site is microservices — the frontend survived long enough to tell you the backend died.

Which is easier to *start*?

🎤 SAID IN CLASS

Room: *"Monolith."* **Instructor:** *"Of course — you just do* `npm start` *and it's done."*

Microservices? Many components, and you have to bring **each and every one** up. Real friction on day one.

The board she drew

Monolith	Microservices
✅ Simple to start	⚠️ More operational complexity
⚠️ Complex, delayed deployment — *"even if there's only one component, it's very hard to deploy one thing"*	✅ Easily deployable
One common codebase	Micro-components, one per feature
Tightly coupled — something breaks → whole website down	**Loosely coupled** — components don't depend on each other

😄 CLASS MOMENT

Live moment: a student confidently put "easily deployable" in the *monolith* column. Instructor: *"Really? Don't you think this is wrong? Which one is easily deployable?"* → moved to microservices. Then: *"I was just checking — you sleeping or what?"*

◆ ◆ ◆

06 🌉 Why DevOps Even Exists

The six-month product question

> **Q:** You have to build a real product — a placement portal, a game, a portfolio site — in **six months**. Monolith or microservices?

The room went quiet. So she answered it herself:

🎤 SAID IN CLASS

"That's the reason DevOps comes into the picture."

Microservices + DevOps doesn't just fit in six months — it ships in **three to four**. Here's how each tool removes a specific pain:

Problem 1 — "how do I even ship 6 separate components?" → **Docker.** Every component (frontend, backend, user service, payment service) gets its own image. One file, one image, and you can **pull it from anywhere in the world** — you fetch it by a single **SHA line**. Problem solved.

Problem 2 — "how do I deploy fast, and scale without dying?" → **CI/CD.** The bad way:

```
one server → dump code → npm start
need more? → new server → git clone → dump code again → npm start
need more? → repeat forever...
```

> ⚠️ **WATCH OUT**
>
> *"It will not work like that."*

The good way: **push code → pipeline updates it → pipeline deploys it.** Tools: **Jenkins** and other CI services. You never run `npm install` or `npm start` by hand again — it's all written inside the pipeline. Your entire job becomes:

```
git add .
git commit -m "..."
git push
```

Problem 3 — "where does it all actually run?" → **Terraform**, holding your cloud credentials, automatically provisioning the server or Kubernetes system it deploys onto.

> ✅ **LOCK THIS IN**
>
> **Her reality check about real jobs:** *"Once you enter a company and check their codebase, they're doing exactly the same thing. They're not manually deploying — they add code, they push, the CI/CD does everything."*

What the word means

DevOps = Dev + Ops

Half	Who	What they do
Dev	Developers	write the code
Ops	Operations	deploy and run it

🎤 SAID IN CLASS

"Dev means developer — of course, common sense. Ops means operational." And DevOps is what happens when those two stop throwing work over a wall at each other.

◆ ◆ ◆

07 🧰 The DevOps Toolbox — All 13 Tools

She built this list live with the class, one tool at a time. This is the syllabus of your entire career, so it's worth knowing in order.

1️⃣ Git & GitHub

Where the code lives. First tool on the board, no debate.

2️⃣ Docker

🎤 SAID IN CLASS

"Docker is the base."

Everything after this assumes containers.

3️⃣ A programming language

Python first. That's what you focus on now.

Go second — and here's the question she ambushed the class with: *"Docker, Kubernetes, Terraform, IAM, SQL tools — what language are they all built in?"* → **Go.** That's why Go is the second preference: the DevOps tools themselves are written in it.

💡 GOOD TO KNOW

Don't just say "Go is trending because LinkedIn said so." Know *where* it's used: the tooling layer.

4️⃣ Linux

🎤 SAID IN CLASS

"Linux is the core of DevOps. You can say the heart of DevOps."

5️⃣ Networking

What you actually need:

- **OSI layers** — application, presentation, session, transport, network
- **TCP/IP**, **TCP vs UDP**

🧠 PICTURE THIS

What the OSI layers actually are — because she lists them and nobody explains them.

OSI is a **7-layer stack** describing how your click becomes electricity and back again. Each layer only talks to the one above and below it, so each can be swapped out independently — that's the entire point of the model.

Read it **bottom-up**, because that's how data physically travels:

#	Layer	Plain-English job	You'd recognise
7	**Application**	the thing you actually asked for	HTTP, HTTPS, DNS, SSH
6	**Presentation**	translate/encrypt/compress the data	TLS, JPEG, UTF-8
5	**Session**	open, hold and close a conversation	login sessions
4	**Transport**	split into packets; deliver reliably (or not)	**TCP**, **UDP**, port numbers
3	**Network**	find a route across the internet	**IP**, routers
2	**Data Link**	move a frame to the next machine on this cable	**MAC** addresses, switches
1	**Physical**	actual electricity, light, radio	cables, Wi-Fi

🧠 **The posting-a-parcel analogy:** you write the letter (**7**), put it in a standard envelope (**6**), agree with the recipient you're corresponding (**5**), split a big order into several boxes and number them (**4**), write the destination address so the postal network can route it (**3**), hand it to the local van going to the next depot (**2**), and the van drives on actual roads (**1**).

✅ **The mnemonic everyone uses, top-down: A**ll **P**eople **S**eem **T**o **N**eed **D**ata **P**rocessing.

💡 **Why you're told it matters in DevOps:** debugging is layer-hunting. `ping` fails → layer 3. Ping works but the port refuses → layer 4. Port opens but you get a 502 → layer 7. Naming the layer tells you which tool to reach for next.

- **Networking commands** — and she drilled these as a debugging story:

Situation	Command
Website is down — first thing you try	`ping` (you get bytes back → host is alive)
Ping worked, but is the *route* healthy?	`traceroute`
App should be on port 8080 — can I reach that port remotely?	`telnet`
What's listening on this box?	`netstat`

⚠️ WATCH OUT

She specifically rejected one answer: a student suggested `curl`. Her response: *"Curl is a Windows command, we're not using that for this"* — she wanted **`traceroute`** and **`telnet`** for connectivity debugging.

🔍 **Simplified, since this one can confuse you later:** `curl` is not Windows-only — it ships on Linux and macOS too, and it's a great tool for testing whether an HTTP endpoint responds. What she meant is that `curl` tests the *application layer* ("does this URL answer?"), whereas `telnet` / `traceroute` / `netstat` test the *network layer* ("can I even reach this port, and where does the traffic die?"). For port-level debugging — which was the question — `telnet` is the right answer. In her course, answer `telnet` / `traceroute` / `netstat`.

6️⃣ CI/CD

CI = Continuous Integration. No trick there.

CD = two different things, and this is the interview trap:

	Meaning	Approval?
Continuous Delivery	pipeline runs, artifact is ready, then it **waits for a human**	✅ A developer reviews the PR, approves it, *then* it goes to production
Continuous Deployment	pipeline runs and it just **goes live**	❌ No manual approval

Why delivery exists at all: *"Production is the one thing clients are directly using. We can't directly push everything into production."*

A full pipeline, as she described it:

```
git push
  ↓  CI
install Node.js  →  npm install  →  npm run build  →  npm start
  ↓  CD
build Docker image  →  deploy to server
  ↓
(if it pauses for approval → Continuous Delivery)
(if it just deploys      → Continuous Deployment)
```

🎯 INTERVIEW ALERT

Interview alert: *"Sometimes in interviews they ask how much depth you have in CI/CD. It's a basic question."* Knowing delivery-vs-deployment is the answer they're fishing for.

7️⃣ Cloud

Provider	Note
AWS	the major one — start here
Azure	strong second, also a good start
GCP	*"how can we forget Google"*
Oracle	exists, also explored

> 💡 GOOD TO KNOW
>
> **Her practical tip:** start with whichever one you have **free credits** for.

8️⃣ Kubernetes (K8s)

Fun fact first: why "K8s"? Because **K-ubernete-s** has **8 letters** between the K and the s. A student got it instantly.

Why you need it — Reason 1: scaling.

Picture your microservices app with 4 services (user, payment, cart, notifications), deployed with Docker + CI/CD. It handles **2,000 users/hour**. Now imagine you're Netflix during an **IPL match** — lakhs, crores, *billions* of viewers hitting play at once.

The manual way to scale:

```
new server → push code → route traffic → repeat
new server → push code → route traffic → repeat
new server → push code → route traffic → repeat
...IPL ends...
now manually scale ALL of them back down
```

Kubernetes just does it. Scale up **and** scale down, automatically. *"We don't need to write any code for scaling up and down. We just run one Helm command, that's it."*

Reason 2: managing containers. (A student got this one.)

Reason 3: self-healing. This was her favourite scenario:

> 🧠 PICTURE THIS
>
> **The 3 a.m. scenario:** You scaled to 5 servers. Traffic is flowing. Suddenly the **main server** — the one everything is routed through — dies. In production. Or worse: your main server is in **Asia**, you scaled into the **US**, you're mid-redirect, and the Asia server fails.
>
> **Kubernetes' answer:** the pod restarts itself. A new pod comes up automatically in your service. No human wakes up.

The Kubernetes vocabulary she introduced:

Term	Plain-English meaning
Pod	*"nothing but a Docker container"*
ReplicaSet	handles scale up / scale down by itself
StatefulSet	used for **MySQL / databases** — things that must remember who they are
Volume	where your data survives; how backups and replicas work
Helm	*"one of the package managers for Kubernetes"* — see below
`kubectl scale down`	how easy scaling actually is

🧠 PICTURE THIS

"Package manager for Kubernetes" — what that actually means.

You already know package managers: `apt install nginx` on Ubuntu, `npm install express` in Node. You don't download source, compile it, and wire up config files — you say the name and it handles the rest.

Helm is that, for whole applications on a cluster. Deploying something real on Kubernetes by hand means writing several YAML files (a Deployment, a Service, a ConfigMap, maybe a StatefulSet and a volume claim). Helm bundles all of them into one reusable package called a **chart**, with the values you'd want to change pulled out into a single settings file.

```
helm install my-db bitnami/mysql --set auth.rootPassword=secret
#    └─ one command instead of five YAML files
```

✅ **So her "we just run one Helm command and that's it"** is literally true: `helm upgrade --set replicaCount=10` scales the whole app without you touching a single YAML file.

The data-safety story: a realistic app might run 1 frontend container, 3 backend containers, and 3–4 database containers to fetch and **back up** data.

🧠 PICTURE THIS

Has this happened to you? You log into a website and your credentials/data are just… gone. *"Nowadays it doesn't happen — because they're already using Kubernetes."* Your data sits in **multiple locations** — Asia region, US region, Ohio region — as replicas. That's what Kubernetes volumes buy you.

What came before: Docker Swarm. *"Previously we had a Docker Swarm cluster. But there were some problems in that Docker Swarm cluster — that's why we need Kubernetes."*

Who built it: Google.

9️⃣ Terraform

🎤 SAID IN CLASS

One-word definition from a student: *"Managing infrastructure."* → correct.

The question Terraform answers: you've written the code, containerised it, built the pipeline, set up Kubernetes… **but where is the cluster?** Somebody has to actually create it.

Cloud	Managed Kubernetes service
AWS	**EKS** — Elastic Kubernetes Service
Azure	**AKS** — Azure Kubernetes Service
GCP	**GKE** — Google Kubernetes Engine

Life without Terraform — for *one* application you need a server, a VPC (networking), a database (RDS), an S3 bucket… so you click:

```
Open AWS console → EC2 → choose an image (Linux? Windows?)
  → create a PEM / secret key → pick which ports to open
  → pick instance type (t-series, micro, nano...) → Create
Then repeat all of that for VPC.
Then repeat all of that for S3.
Then repeat all of that for RDS.
```

Now do it **ten times** for ten servers in the Asia region. 😵

Life with Terraform:

```
terraform init     # get ready
terraform apply    # go
```

One file. Your credentials. And it creates **EC2, VPC, S3, and RDS in parallel**.

🎤 SAID IN CLASS

"You just write a file and execute that file, that's it. You don't need to click anything inside the AWS or Azure portal."

Category name: infra provisioner (Infrastructure as Code).

🔟 Ansible

The task: install Docker on **10,000 servers** at once.

Without Ansible:

```
SSH into server 1 → run the 4 Docker install commands → verify
SSH into server 2 → run the 4 Docker install commands → verify
... 10,000 times ...
```

With Ansible: one file. In it you put: - the **server names / hostnames** - the **SSH key** to connect with - the **4 commands** you want run

Run the file once → Docker lands on all 10,000 servers.

✅ LOCK THIS IN

Milestone she called out: *"If you've learned Docker, CI/CD, Linux, GitHub, networking, some database queries, then Docker, Kubernetes and Ansible — you've covered almost 80% of DevOps."*

1️⃣1️⃣ Monitoring

Your app is live. Now what? *"Monitoring. Health check. Testing."*

Tool	What it actually is
Grafana	the **dashboard** — the pretty graphs
Prometheus	the thing that **collects** the metrics Grafana draws
Sentry	mentioned by a student (heard as "Sanity" in the recording)

🎤 SAID IN CLASS

"These are the two main tools we're currently using in the industry" — Grafana + Prometheus — *"but this is also old now. GitOps came after."*

1️⃣2️⃣ GitOps

🧠 PICTURE THIS

The scenario: It's **Sunday, midnight.** Something crashed on your website. How do you find out what happened — and make sure it never happens again?

Room: *"We'll check the logs."* Correct first move. But logs tell you *what* broke, not *what changed*.

GitOps tracks everything from **version control all the way to production** — so you can see exactly what happened and stop it recurring.

1️⃣3️⃣ DevSecOps (Security)

🎤 SAID IN CLASS

"Don't you think security is the main concern? Don't you think?"

DevSecOps = Dev + Sec + Ops. Sec = security, obviously. But the real question: **DevOps is 200+ tools — how do you add security to *each one*?**

SAST — Static Application Security Testing - **Tool: SonarQube** (you can run it straight from Docker). - You spin up SonarQube, add your project, connect your app URL, and it reports the security gaps **in your code**. - Concretely: it reads your `package.json` (Node) or `requirements.txt` (Python) and flags dependencies like *"you're on version 0.x, which was deprecated two years ago — anyone can hack you through this."* - It runs **when you push your code to GitHub**. Nothing is running yet — hence *static*.

DAST — Dynamic Application Security Testing - Your code is **live right now**. A package you depend on gets updated. A base image changes. IBM deprecates a library you're using — while your site is serving traffic. - That's when DAST applies. **Tools: Trivy, Snyk.**

⚠️ WATCH OUT

Don't mix these up: SAST and DAST are **concepts**, not tools. SonarQube is a *tool for* SAST. Trivy and Snyk are *tools for* DAST.

OWASP - *"OWASP is the one thing you need to learn inside DevSecOps."* - It's a **concept/framework** (like networking is a concept), with roughly **10 items** and a tool for each.

Where Trivy actually gets used — Docker image scanning

First, the Dockerfile anatomy she walked through (this is your Class 6 preview):

```
FROM node:20              # base image
WORKDIR /app              # set the work directory
RUN npm install           # install what you need
COPY . .                  # copy the code in
EXPOSE 3000               # expose the port
CMD ["npm", "start"]      # or ENTRYPOINT — start the app
```

Trivy's job: after you build the image from that Dockerfile, Trivy **scans the image**. If your base image — `node:16`, `node:20`, `node:23`, whatever — carries a known vulnerability, Trivy catches it.

🎯 INTERVIEW ALERT

Interview alert, straight from her: *"These are deep concepts. Sometimes in placements these help, because no one knows them. If a college student knows this thing, they think — this person has that kind of knowledge."* Also: *"Even if you know everything in DevOps but not the best practices, and 10 people are competing for one seat, they'll pick the DevSecOps person."*

😄 And when she asked where Trivy is used: ***"ChatGPT-se not poochna."*** (Don't ask ChatGPT.)

◆ ◆ ◆

08 🎒 The CS Fundamentals They'll Grill You On

She paused the tool list for a career detour.

The four subjects asked in *any* interview:

#	Subject
1	**DSA**
2	**Networking**
3	**Operating Systems** — *"which is Linux, nothing but Linux. They'll ask Linux commands."*
4	**SQL / MySQL**

Plus two more she insisted on: - **System design** — *"DSA and system design are a benchmark. You have to cross it."* - **OOP** — *"If you don't know inheritance, polymorphism, abstraction, it won't work."*

And narrowed for DevOps specifically:

✅ LOCK THIS IN

The three you MUST have: Linux · Networking · Databases.

◆ ◆ ◆

09 🧭 The Instructor's Own Career Story

Worth keeping, because it's the honest version of "how do I pick a specialisation."

```
Started as a DEVELOPER
  → ~7 internships during college
  → finished 4th year
  → "Honestly, I didn't like it. It was very boring for me."

Switched to DATA SCIENCE
  → "I don't like data. Eight hours I cannot sit with one CSV file."

Switched to CLOUD
  → "Okay, that's fine. I don't need to code, I don't need to do data. I just
deploy."
  → "I like troubleshooting. I like errors — when an error comes I have to debug it,
     resolve it. In servers, not in websites."

Built a 3-tier full-stack project + Docker + CI/CD + Terraform + Ansible
  → joined a startup
  → the CEO said: "you should explore more security"
  → → DevSecOps.
```

"But won't AI take DevOps jobs?"

Her argument for why DevOps is safe — arguably *safer* because of AI:

🧠 PICTURE THIS

If AI grows, what grows with it? More **cybersecurity** roles. More **data engineer** roles. And all those AI models have to *run somewhere* — on servers that need **DevOps engineers**, **cloud engineers**, and **SREs**.

Those four domains are the currently-trending ones. AI doesn't remove the servers; it adds more of them.

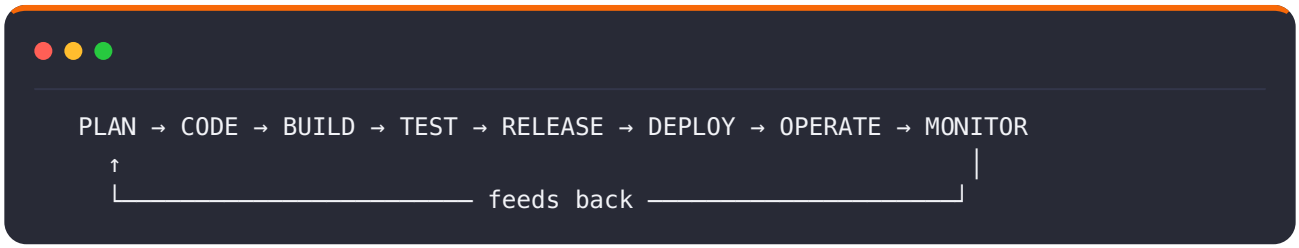
◆ ◆ ◆

10 ∞ The DevOps Lifecycle — 8 Stages in a Loop

🎤 SAID IN CLASS

"You see, my style is I will always ask questions."

She drew the classic DevOps infinity/circle diagram, noting it looks a lot like the SDLC diagram you've already seen.



🗺️ Plan

A student answered *"system design"* — good.

Plain version: **what are you building?** *"You want to build a product like Flipkart? You won't build a system application. You want to build something like Zepto — a shopping application? Then you need a plan: what kind of cards, what kind of clothes, will it look like Amazon or Flipkart?"*

✅ **LOCK THIS IN**

Her logic: *"When you have that plan, then you will code. If you don't have a plan, how will you write code?"*

💻 Code

Write it. Build the components.

🔨 Build

This is your CI/CD pipeline doing `npm install`, `npm start`, building the **Docker image**.

🧪 Test

She asked what you'd test. Room: *"feature testing, unit test, security test, integration test."* All valid — but she wanted the *first* thing:

Level 1 — is it even alive? - Is the **frontend** up? The **backend**? The **DB**? - Does the **landing page** load? - Does **username + password** work? - What if someone enters the **wrong** password? - What if someone launches a **DDoS attack**? - Password rules: at least one **special character**, one **number**, **8–16 characters**

Level 2 — the formal stuff: - **unit tests** - **integration tests** - **SAST** (SonarQube, if you're using it)

🚀 Release

"Have you ever heard the term release? Have you done a release?"

😄 **CLASS MOMENT**

The internship moment: *"Anyone done any internship, worked on a live project?"* → total silence. *"No one did any internship? Poor people. Really? Third year? Really?"* … *"I guess you guys need to do at least one internship in your life. When you work with those developers, you get to know this process."*

What release actually is: a **pre-release phase** before production, where you **cross-verify everything** — then you release, then it deploys.

🧠 PICTURE THIS

Have you noticed this? Flipkart changes its icon during a sale. Google changes its logo for IPL, Diwali, Holi, a football tournament. *"How are they doing that?"* → **pre-release** → **release** → **deploy.** That's the machinery behind a doodle.

📦 Deploy

Now it's actually live and you can see the website.

⚙️ Operate & 📈 Monitor

Run it and watch it — with **Grafana**, **Prometheus**, GitOps tooling.

◆ ◆ ◆

11 💼 Job Roles — The Whole Menu

She listed every role you could aim for, with commentary.

Role	Her notes
DevOps Engineer	*"Easy one, that's it."* The default.
Teacher / Trainer	*"For that you need a lot of patience and passion. Without passion you can't be a teacher."*
System Admin	Red Hat certifications, deep-dive Linux, server troubleshooting. *"That's a good job."*
DevSecOps Engineer	DevOps + security.
Platform Engineer	Trending — *"but it's also old now, like four months back."* 😄
SRE — Site Reliability Engineer	The newest hot one. See §12 for what they actually do.
Cloud Architect	*"One of the old roles."*
Cloud Engineer	Solid, in demand.
Solution Architect	Needs **~5 years** experience and real **system design** skill. *"Even though I have certifications in AWS, Azure and GCP, I'm still not a solution architect."* High-paying — *"after your 30s, not now."*
Prompt Engineer	Yep, on the list.
AIOps Engineer	

Role	Her notes
MLOps Engineer	*"These guys are just transitioning their career from DevOps to AIOps or MLOps. It's very very easy"* — if you know cloud → DevOps → DevSecOps → MLOps/AIOps. Needs **at least 4 years** experience, *"at least, if you're extraordinary."*
ML SecOps	Mentioned in passing.
AI Cloud Engineer	**Recently launched by Microsoft.** Cloud + DevOps + some AI — e.g. *how do you deploy agents in an AI factory?* Related: **Azure AI Foundry**, **AWS Bedrock**, and **Kiro** (*"nothing but Cursor's old version, from AWS"*).
Developer Advocate	Business knowledge + communication with direct clients + technical depth. *"When you have everything, that means you're an SME expert — that's why this role was added."*

💡 GOOD TO KNOW

Lots of openings in DevOps and DevSecOps right now, per her.

◆ ◆ ◆

12 🙋 Q&A — Role Confusion, Cleared Up

"Solution Architect vs. Engineer?"

🎤 SAID IN CLASS

"Solution architect won't do the CI/CD pipeline. Solution architect won't do Kubernetes. Solution architect will not do anything." 😄

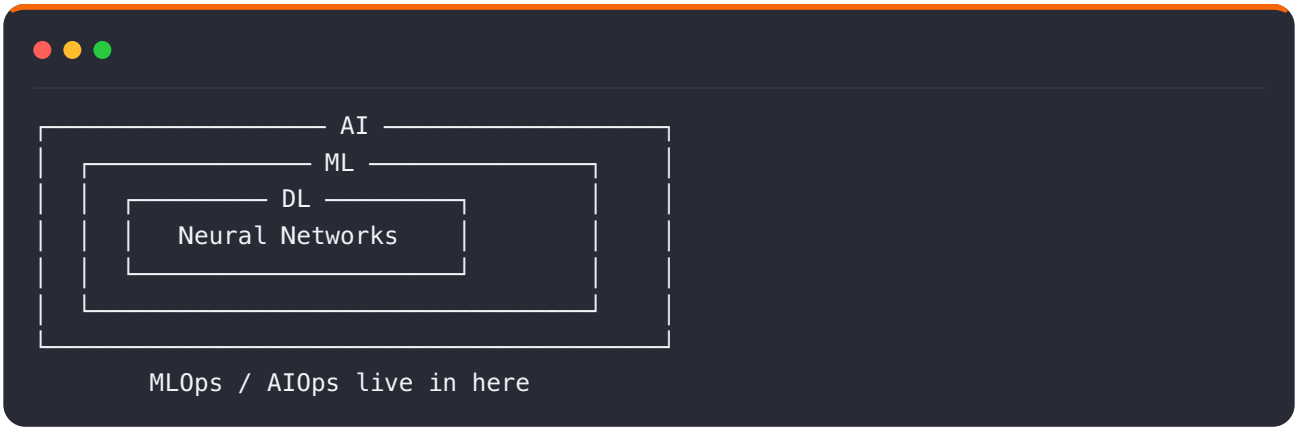
What they **do** do — two things:

1. **Design the complete workflow** for a product: the whole flow, from user login all the way to the deployment stage.
2. **Fit new features into an existing workflow**: *"If you want to add one feature, how will you add that particular feature inside the existing workflow?"*

They sit and architect. They don't wire the pipeline.

The AI / ML / DL circle

She drew the nested-circles diagram (*"I'll try to draw a good circle, hopefully"* 😄):



Role	What they actually do
AI Engineer	Research, plan, and **build the AI agents** overall. *"They are not going to deep dive into algorithms."* They take an existing LLM, deploy it, and build agents on it.
MLOps Engineer	Works **on the algorithm** — enhancing it, creating new ones, checking **space and time complexity**. *"They are actually building those LLM models"* — building, training, deploying.

Who they work with: MLOps engineers sit between **developers** and **DevOps** people, building/enhancing the algorithm, producing the LLM, deploying and training it. After training, the **agent** gets built — that's the AIOps/AI-engineer side.

What does an SRE actually do?

Start with the contrast:

- **DevOps engineer** = works with all the tools — setting up CI/CD, Terraform, Kubernetes, everything. **Purely technical.**
- **SRE** = knows the **business** level *and* the **technical** level.

🧠 **PICTURE THIS**

The full story she told, because it's the clearest explanation of the role:

A **client** wants a feature. They tell the **founder** — who is non-technical. The founder tells the **dev team**: *"add a form that grabs data from users."* The team builds exactly that. They show the client. The client says: ***"I don't want this kind of form data. I want something different."***

So who catches this mismatch before three weeks are wasted? **Someone who understands what the client actually means AND what's technically being built.** That's the **SRE** — and the title for owning that requirement translation is **Program Manager** or, as she put it, ***"Product Owner is the main term we're using right now."***

Also: when something breaks at the *upper level*, the client escalates to the manager, the manager escalates up — and the **SRE** is the person who can talk both languages.

✅ LOCK THIS IN

Simple decision rule:

- Only technical knowledge? → DevOps, DevSecOps, Cloud, Solution Architect.
- Technical **+** business knowledge? → **SRE** (or Developer Advocate).

A student also asked about the **CTO** and **business manager** angle — same theme: once you have a startup, a technical team, a non-technical team, marketing, social media, you *need* someone in the middle who can translate business logic to a client who *"doesn't know the work — they only know what they want."*

◆ ◆ ◆

13 🎮 Quiz Time — Team A vs. Team B vs. Team C

She split the room into **three teams by seating** and ran a rapid-fire relay. Answers had to come from a specific team, and specific rows within a team.

Quick recap she did first

> *"First thing we covered? Monolithic. Second? Microservices — pros and cons. Next? DevOps tools — Docker, Kubernetes, Terraform, Ansible: how they work, where we use them, why we need them. Then DevOps architecture / lifecycle — build, test, deploy, release, operate, monitor. And last? Job roles. Exactly."*

The quiz board

#	Question	Asked to	Answer given	Verdict
1	Which architecture for a **college placement application**?	Team A	*(first answer was monolith)*	❌ → correct answer: **Microservices**
2	Name **6 DevOps tools**	Team B (row by row)	Docker, Kubernetes, Grafana, Prometheus, Linux, Terraform, GitOps, Cloud	✅ (with one disqualification — see below)
3	Name **2 job roles** each	All three teams	**A:** DevOps · **B:** SRE, Solution Architect, AIOps/MLOps, Cloud Engineer, ML SecOps · **C:** Platform Engineer, AI Cloud Engineer	✅ *"Platform engineer — best one."*
4	Where do we use **Jenkins**?	Team A	**CI/CD**	✅ *"Perfect."*

#	Question	Asked to	Answer given	Verdict
5	Where do we use **Trivy**?	Team B, first row	first: "security checks" (*"nearby"*), then: **Docker image scanning**	✅ *"Perfect."*
6	Which tool do we use for **SAST**?	Team B, third row	**SonarQube**	✅ *"Right one."*
7	🔥 **Hard Linux:** which command to troubleshoot a **specific port** on a server?	Team B, last column	student said `ls`	❌ → *"That's a normal command."* Correct: `telnet`, `traceroute`, `netstat`
8	🟢 **Easy:** which is the major cloud platform?	—	**AWS**	✅ *"It's not fair"* 😄
9	Where do we use **Ansible**?	Team A	took a while, eventually got there	✅ **Configuration management** — installing/configuring across many servers
10	🏆 **What does DevSecOps mean — in ONE word?** (not "security")	Team B, then open to all	tries: "security" ❌ · "secured DevOps" ❌ · "integrating security" ❌ · "IaC" ❌ · "safeguarding" ❌ → then: **"SHIFT LEFT"**	✅🎉 *"Who told this? Nice. Perfect. Clap for her, actually."*

😄 **CLASS MOMENT**

Quiz drama: Team B answered "Kubernetes" for Q2 — but out of turn. *"Your time was not there, only two tools… Kubernetes gone."* Ruthless.

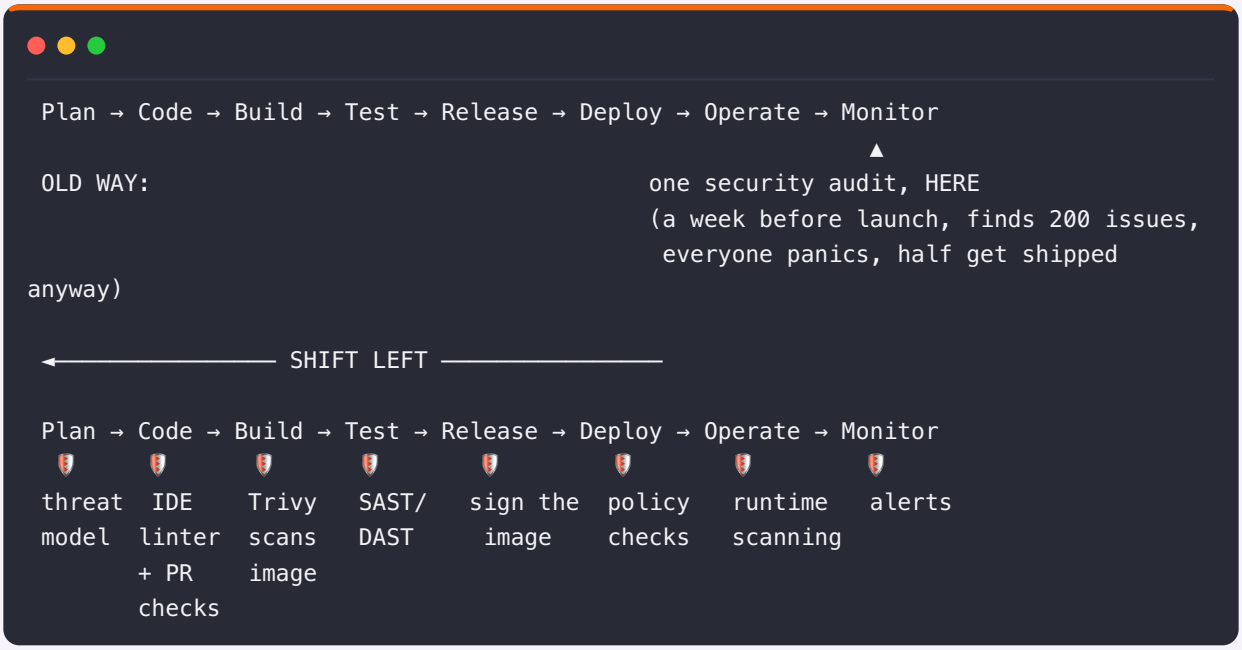
✅ **LOCK THIS IN**

The one-word answer worth tattooing on your notes: DevSecOps = SHIFT LEFT. Move security from the end of the pipeline to the *beginning* — into the IDE, the PR, the image build, the running cluster.

🧠 PICTURE THIS

Why the word is "left" — picture the pipeline drawn on a whiteboard.

Pipelines are always drawn left-to-right, so "shifting left" means "moving it earlier."



✅ **The reason it's cheaper:** a vulnerability caught by a linter as you type costs you 10 seconds. The same vulnerability caught in production costs an incident, a patch, a release, and possibly a disclosure. Same bug, wildly different price — and the price rises the further right you find it.

◆ ◆ ◆

14 🧱 The Great Monolith Debate

Question 1 of the quiz turned into the best discussion of the class. Here's the whole argument chain.

Step 1 — What components does a placement portal need?

She built the list with the class. This is *why* the answer is microservices:

#	Component	Why it's separate
1	**User data / login**	*"You want to be placed — what do you want to see first? Your data, login."*
2	**Company data**	because you want to apply to companies
3	**Resume upload**	needs its own component
4	**Application tracking**	*"You apply, then you need to track your application"*

#	Component	Why it's separate
5	**Interview rounds**	OA round → DSA round → 1–2 technical rounds
6	**HR round**	HR details, HR policies, company data — *"totally, completely different from the technical round. In a technical round they say 'write a Dockerfile' or 'write an algorithm'."*

→ **six to seven features.** That's a microservices shape, not a monolith shape.

Step 2 — "But what if it's a monolith?"

> **Q:** 10,000 students all apply for one job role. Your monolith handles 8,000 users. It goes down. What now? **Room:** *"Scale up."* **Instructor:** right — but **scaling a monolith is very hard.** It's one large application with one large dataset. You have to spin up another server, redirect the traffic, and that takes *"a huge time."*

The real-world receipt: *"Previously Amazon had a monolithic architecture. Because of that, their application crashed — it was down during a streaming event. Then they broke everything down and moved from monolith to microservices."*

Step 3 — "Doesn't it depend on scale?" (great student question)

> **Student:** the architecture depends on the number of users, right? If there are only 100 users…
> **Instructor:** *"Yeah — if you have only 100 users for your internal team, then use monolith. Why not?"*

But then the trap:

🧠 PICTURE THIS

"The question said college placement application. That means you're going to be alumni. New students will come, you will go. So student data will always grow. Nobody removes the data — alumni data is also there. And if user data always grows, then we can't use monolith."

That's the tell. The words "college placement" are doing the architectural work.

Step 4 — "But microservices cost more money"

> **Student:** microservices have more expenses. **Instructor:** *"Yes."* So the honest answer is:

```
Initially → build with monolith. It's cheaper and faster to start.
Later     → if you don't want to lose your money, go microservices.
```

⚠️ WATCH OUT

The catch nobody tells you: migrating later is brutal. *"You already wrote your code. Now you need to detach the frontend, the backend, every piece, then restart everything — and there's downtime. And you can't afford downtime in business."*

Step 5 — "Why not just scale the data in a monolith?"

The sharpest question of the class, asked twice from different angles.

The answer — single point of failure:

🎤 SAID IN CLASS

"If a single failure happens, your whole application is down. That means your data will be lost. That is the only concern."

	Monolith	Microservices
Where data lives	**one** server — everything in one place	**different** servers / DB instances
One thing breaks	**everything** breaks	that one thing breaks

Now picture it with Kubernetes:

```
EKS cluster
├── 2 containers  → frontend
├── 5 containers  → backend
└── containers    → connecting to your DB (RDS instance)
```

Scaling that, or routing traffic from one container to another, is **easy**. In a monolith you'd have to figure out which server, add another database, wire it all together — and *"then it will create a latency issue. Because of that latency issue itself, we use EKS."*

✅ LOCK THIS IN

The punchline: *"With Kubernetes you just do* `kubectl scale down` *. That's it."*

Step 6 — "So is it 60/40? Which 40% is still monolith?"

Student: not 90% microservices — maybe 60%. So which kinds of projects are still the other 40% monolith?

Instructor: *"That's kind of a political question"* 😄 — then gave a real story:

🧠 PICTURE THIS

The Microsoft story. A friend at Microsoft, asked this exact question at an event, said: their application is written in **C**. Not even C++ — old C. **Millions of lines.** They'd have to figure out who wrote all of it, understand all of it, and then re-cut it into microservices. So they stay monolith.

The rule that follows: ***"What is working, just take it. Don't touch it. That's the first rule."***

And for you, right now:

🎯 INTERVIEW ALERT

Interview alert: *"In an interview they'll ask what microservices architecture is and how you use it. But normally what they'll do is look at your resume — 'this person made a full-stack, three-tier application with the container approach.' That's what they want, because that's the same thing they're doing. If you build your full-stack application as a monolith, it won't work — it's an old approach."*

🧠 PICTURE THIS

"Three-tier application" — the phrase she keeps using, defined.

It's just the three jobs every web app has, kept in three separate pieces:

```
TIER 1 — PRESENTATION   the frontend. What the user sees.       React, HTML, nginx
      ↕
TIER 2 — APPLICATION    the backend. The rules and the logic.   Node, Java, Python
      ↕
TIER 3 — DATA           the database. Where things are kept.    MySQL, Postgres,
MongoDB
```

✅ **And "with the container approach" means one container per tier.** That's exactly the app you build across Classes 8, 9 and 10 — frontend + backend + MySQL, three containers, wired together. So when a recruiter scans your CV for "three-tier full-stack with containers", you'll have literally built it.

💡 **Why "tier" and not "microservice":** three tiers is a *layering* choice (who does what). Microservices is a *splitting* choice (how many independently deployable pieces). You can have a three-tier monolith, and you can have twenty microservices inside the application tier alone.

New startup project? **Microservices, surely.**

◆ ◆ ◆

15 🔁 Class Recap Checklist

Tick these off — this is literally her own recap:

✓ **Monolithic architecture** — what it is, why one change redeploys everything

✓ **Microservices architecture** — pros and cons

✓ **The 13 DevOps tools** — what each does, where it's used, why you need it

✓ **DevOps lifecycle / architecture** — plan → code → build → test → release → deploy → operate → monitor

✓ **Job roles** — and which one fits which skill mix

◆ ◆ ◆

16 📔 Cheat-Sheet — Everything Introduced Today

Concepts

Term	One-line meaning
Monolith	All features in one codebase, deployed as one unit. Tightly coupled.
Microservices	Each feature its own codebase and deploy. Loosely coupled.
Tightly coupled	One thing breaks → everything breaks
Loosely coupled	Components don't depend on each other
API	The agreed way one program asks another for something — see the note in §16
DevOps	Dev (write code) + Ops (deploy & run) — automated so microservices are manageable
SDLC	Software Development Life Cycle
CI	Continuous Integration — build & test on every push
CD (delivery)	Ready to ship, but **waits for human approval** before production
CD (deployment)	Ships to production **automatically**
IaC	Infrastructure as Code — your servers defined in a file
SAST	Static Application Security Testing — scan the **code**
DAST	Dynamic Application Security Testing — scan the **running** app
OWASP	Security concept/framework, ~10 items, tools for each
DevSecOps	Security at every stage — **"shift left"**
3-tier app	Presentation (frontend) + Application (backend) + Data (database)
GitOps	Track everything from version control → production
K8s	Kubernetes — 8 letters between K and s 🙂
Pod	*"Nothing but a Docker container"*

Term	One-line meaning
ReplicaSet	Auto scale up / scale down
StatefulSet	For databases (MySQL) that must keep identity
Volume	Where data persists / replicates
Helm	Package manager for Kubernetes
502 / 503	Gateway error → backend problem
Single point of failure	Monolith's fatal flaw

Tools

Tool	Category	Job
Git / GitHub	SCM	store & version code
Docker	Containers	*"the base"* — package each component into an image
Docker Swarm	Orchestration (legacy)	pre-Kubernetes clustering; had problems
Kubernetes	Orchestration	auto scale, manage containers, self-heal. Built by **Google**
Jenkins	CI/CD	run the pipeline
CircleCI / GitHub Actions	CI/CD	alternatives
Terraform	IaC / infra provisioner	create cloud resources from a file
Ansible	Config management	run the same commands across N servers
Prometheus	Monitoring	collect metrics
Grafana	Monitoring	dashboard the metrics
Sentry	Monitoring	error tracking (student mention)
SonarQube	Security (SAST)	scan code + dependencies
Trivy	Security (DAST)	**scan Docker images**
Snyk	Security (DAST)	dependency/vuln scanning
Python	Language	#1 for DevOps
Go	Language	#2 — Docker, K8s, Terraform are written in it

🧠 One term everyone uses and nobody defines: API

🧠 PICTURE THIS

An API is a menu.

You sit in a restaurant. You don't walk into the kitchen and start cooking — you read a **menu** of things you're allowed to order, say one out loud, and food arrives. You never find out how the kitchen works, and the kitchen can be completely rebuilt without changing your experience of ordering.

An API (Application Programming Interface) is that menu, between two programs.

```
Your frontend  ──►  GET /api/jobs           "list the open jobs"
               ◄──  [{ "id": 1, "title": "SDE Intern" }, …]

Your frontend  ──►  POST /api/applications  "apply to job 1, here's my resume"
               ◄──  { "status": "submitted" }
```

✅ **Why it's the thing that makes microservices possible.** Each service publishes a menu and hides its kitchen. The payment service doesn't know whether login is written in Java or Go — it just calls `POST /api/verify-token` and gets an answer. That's the "loosely coupled" property from §5, made concrete.

💡 **And it's why `docker pull nginx` is described as "an API call" in Class 6.** Your terminal doesn't reach into the Docker daemon — it orders from the daemon's menu.

Cloud service name-mapping

Purpose	AWS	Azure	GCP
Managed Kubernetes	**EKS**	**AKS**	**GKE**
Managed AI platform	**Bedrock**	**AI Foundry**	—

Commands mentioned

Command	Use
`chmod`	change file permissions — *"a command you use every single day"*
`ping`	is the host alive?
`traceroute`	where is my traffic dying?
`telnet`	can I reach this specific **port** remotely?
`netstat`	what's listening on this server?

Command	Use
`npm install` / `npm start` / `npm run build`	build & run a Node app
`git add` / `git commit` / `git push`	*"once you have CI/CD, this is your whole job"*
`terraform init`	initialise Terraform
`terraform apply`	create all the infra, in parallel
`kubectl scale down`	scale a Kubernetes service down
Helm commands	scale up/down without writing scaling code

Dockerfile skeleton (preview of Class 6)

```
FROM node:20              # base image
WORKDIR /app              # working directory
RUN npm install           # install dependencies
COPY . .                  # copy your code in
EXPOSE 3000               # expose the port
CMD ["npm", "start"]      # or ENTRYPOINT — start the app
```

◆ ◆ ◆

17 🗒️ Notes on the Transcript Itself

- **Instructor name:** the transcript header credits *"Ritesh Prajapati"* — that's the **developer of the Scaler++ Chrome extension** that downloaded the file, not the teacher. The instructor is **Nensi Ravaliya** (@Nency-Ravaliya), a **DevSecOps Engineer at Standard Chartered**. Classes 2, 3 and 8 confirm the name from the audio and from live terminal paths.
- The recording is auto-generated (Scaler++ Chrome Extension) and **very** ASR-noisy — the first few minutes are pure setup chatter, and the file ends with a long run of *"Thank you. Thank you. Thank you."* which is transcription filler, not content.
- Stray **Hindi** fragments appear where the audio caught side-conversations (e.g. the screen-share negotiation at the start, and *"ChatGPT-se not poochna"* during the Trivy question — meaning **"don't ask ChatGPT"**). These are folded into the relevant sections above with their meaning intact.
- A few names came through garbled and are corrected here from context: **"Sanity" → Sentry**, **"SST" → SAST**, **"SonarQ / Sonargyu" → SonarQube**, **"Trivi" → Trivy**, **"Sniq" → Snyk**, **"CICT / CIICD / CHD" → CI/CD**, **"AXIBLE / MindSybal" → Ansible**, **"Kero" → Kiro**, **"press route / Trashroot" → traceroute**, **"Netstit" → netstat**, **"R&B thing" → Ansible**, **"desktop box and envelopes" → DevSecOps and DevOps**, **"EA Cloud" → AI Cloud (Engineer)**, **"the angel" → likely a community/ angel network**, **"top 500 fortune conference" → Fortune 500 company**.

- The instructor's whiteboard (iPad) content is reconstructed here from what she narrated while drawing — the monolith-vs-microservices pros/cons table (§5), the DevOps lifecycle circle (§10), and the AI ⊃ ML ⊃ DL ⊃ NN circles (§12).